

Machine Learning Lab Assignment 1

Apurba Koirala

Reg No: 22BCE3799

Lab Slot: L25 + L26

Guided by: Dr. Sudha S

✓ Numpy Practice Problems

✓ 1. Create an array of 6 zeros

```
import numpy as np
np.zeros(6)

↻ array([0., 0., 0., 0., 0., 0.])
```

✓ 2. Create an array of 6 ones

```
np.ones(6)

↻ array([1., 1., 1., 1., 1., 1.])
```

✓ 3. Create an array of 6 fives

```
np.full(6, 5)

↻ array([5, 5, 5, 5, 5, 5])
```

✓ 4. Create an array of integers from 1 to 99

```
np.arange(1, 100)

↻ array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
        18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34,
        35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51,
        52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
        69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85,
        86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

✓ 5. Create an array of all the odd integers ranging from 1 to 99

```
np.arange(1, 100, 2)

↻ array([ 1,  3,  5,  7,  9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33,
        35, 37, 39, 41, 43, 45, 47, 49, 51, 53, 55, 57, 59, 61, 63, 65, 67,
        69, 71, 73, 75, 77, 79, 81, 83, 85, 87, 89, 91, 93, 95, 97, 99])
```

✓ 6. Create a 2X2 matrix filled with values from 1 to 4

```
np.arange(1, 5).reshape(2, 2)

↻ array([[1, 2],
        [3, 4]])
```

✓ 7. Create a 3X3 matrix filled with values from 9 to 17

```
np.arange(9, 18).reshape(3, 3)

↻ array([[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]])
```

✓ 8. Make an identity matrix of 4X4

```
np.eye(4)

↵ array([[1., 0., 0., 0.],
         [0., 1., 0., 0.],
         [0., 0., 1., 0.],
         [0., 0., 0., 1.]])
```

✓ 9. With the help of NumPy generate a random number in between 0 to 1

```
np.random.rand()

↵ 0.2673766750983305
```

✓ 10. Compare two 3D arrays and display the results in terms of true and false

```
a = np.random.rand(2,2,2)
b = np.random.rand(2,2,2)
a == b

↵ array([[[False, False],
         [False, False]],
        [[False, False],
         [False, False]]])
```

✓ 11. Create a null vector of size 20 where the 6th value should be 0

```
arr = np.empty(20)
arr[:] = np.nan
arr[5] = 0
arr

↵ array([nan, nan, nan, nan, nan, 0., nan, nan, nan, nan, nan, nan, nan,
        nan, nan, nan, nan, nan, nan])
```

✓ 12. Reverse an array of size 100 using numpy

```
np.arange(100)[::-1]

↵ array([99, 98, 97, 96, 95, 94, 93, 92, 91, 90, 89, 88, 87, 86, 85, 84, 83,
        82, 81, 80, 79, 78, 77, 76, 75, 74, 73, 72, 71, 70, 69, 68, 67, 66,
        65, 64, 63, 62, 61, 60, 59, 58, 57, 56, 55, 54, 53, 52, 51, 50, 49,
        48, 47, 46, 45, 44, 43, 42, 41, 40, 39, 38, 37, 36, 35, 34, 33, 32,
        31, 30, 29, 28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15,
        14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

✓ 13. Find the minimum and maximum values of a 20X20 array using numpy

```
arr = np.random.rand(20,20)
arr.min(), arr.max()

↵ (np.float64(0.00015785740208229537), np.float64(0.9942845503412543))
```

✓ 14. Find mean value of a randomly generated array of size 50

```
arr = np.random.rand(50)
print(arr)
print(arr.mean())

↵ [0.31798318 0.41426299 0.0641475 0.69247212 0.56660145 0.26538949
0.52324805 0.09394051 0.5759465 0.9292962 0.31856895 0.66741038
0.13179786 0.7163272 0.28940609 0.18319136 0.58651293 0.02010755
0.82894003 0.00469548 0.67781654 0.27000797 0.73519402 0.96218855
0.24875314 0.57615733 0.59204193 0.57225191 0.22308163 0.95274901
0.44712538 0.84640867 0.69947928 0.29743695 0.81379782 0.39650574
0.8811032 0.58127287 0.88173536 0.69253159 0.72525428 0.50132438
0.95608363 0.6439902 0.42385505 0.60639321 0.0191932 0.30157482
0.66017354 0.29007761]
0.5133160929276026
```

- ✓ 15. Create an array of size 10X10 with 10 elements valued as nan.

```
arr = np.random.rand(10,10)
nan_indices = np.random.choice(100, 10, replace=False)
arr.ravel()[nan_indices] = np.nan
arr
```

```
array([[0.58044293, 0.12008175, 0.57061716, 0.7137509 , 0.85859351,
        0.35280139, 0.93731513, 0.72129879, 0.23166047, 0.00572598],
       [0.11689188, 0.96798096, 0.56921905, 0.28013314, 0.9310203 ,
        0.98286739, 0.41852343, 0.56001531, 0.36773547, 0.47011685],
       [ nan, 0.74431185, 0.1465173 , 0.59841422, 0.7679763 ,
        0.25234088, 0.34382863, 0.02064424, 0.20061719, nan],
       [0.0622662 , 0.35951365, 0.60351369, 0.84118753, 0.03300218,
        nan, 0.15948705, 0.09049267, 0.42703959, 0.01287134],
       [0.78346142, 0.82474739, 0.15133492, 0.33864228, 0.39879829,
        0.73570685, nan, 0.88358999, 0.877192 , 0.96354178],
       [0.28164217, 0.51958562, nan, 0.55930888, 0.19288205,
        0.96546776, nan, 0.84848533, 0.11458112, 0.56793253],
       [0.52858685, 0.43656571, 0.84629846, 0.19921293, 0.41111958,
        0.41956693, 0.01241146, nan, 0.04892286, 0.24600064],
       [0.67231783, 0.03977114, 0.87227742, 0.77073168, 0.32928457,
        0.86891545, nan, 0.61186492, 0.72203585, 0.23478701],
       [0.99600123, 0.81172163, 0.82051473, nan, 0.39198565,
        0.8098301 , 0.48810569, 0.53530992, 0.75622085, 0.32671117],
       [0.94520915, 0.97231928, 0.03256691, nan, 0.27717346,
        0.98539784, 0.80165977, 0.49423626, 0.74891725, 0.11384221]])
```

✓ Pandas Practice Problems

- ✓ 1. Create a pandas Series using lists and dictionaries?

```
import pandas as pd
s1 = pd.Series([1, 2, 3, 4])
s2 = pd.Series({'a': 1, 'b': 2})
s1, s2
```

```
(0    1
 1    2
 2    3
 3    4
 dtype: int64,
 a    1
 b    2
 dtype: int64)
```

- ✓ 2. Create series using NumPy functions in Pandas?

```
s = pd.Series(np.arange(5))
s
```

```
0
0 0
1 1
2 2
3 3
4 4

dtype: int64
```

- ✓ 3. Find index and values of series in Pandas?

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
s.index, s.values
```

```
(Index(['a', 'b', 'c'], dtype='object'), array([10, 20, 30]))
```

- ✓ 4. How to specify an index while creating Series in Pandas? Find Size and Shape of a Series in Pandas?

```
s = pd.Series([10, 20, 30], index=['x', 'y', 'z'])
s.size, s.shape
```

```
(3, (3,))
```

5. Find the first or last few rows from a Series in Pandas?

```
s.head(2), s.tail(2)
```

```
(x    10
 y    20
 dtype: int64,
 z    30
 dtype: int64)
```

6. Checking the duplicate rows and columns using pandas.

```
df = pd.DataFrame({
    'A': [1, 2, 2, 4],
    'B': [5, 6, 6, 8]
})
print(df)
df.duplicated(), df.T.duplicated()
```

```
(0    False
 1    False
 2     True
 3    False
 dtype: bool,
 A    False
 B    False
 dtype: bool)
```

7. Demonstrate possible missing value analysis approaches using any real world data

```
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True)
df = iris.frame

import random
for _ in range(10):
    i = random.randint(0, df.shape[0]-1)
    j = random.randint(0, df.shape[1]-1)
    df.iat[i, j] = np.nan
df.isnull().sum()
```

```
0
sepal length (cm)  1
sepal width (cm)   3
petal length (cm)  3
petal width (cm)   0
target            3

dtype: int64
```

8. Normalize height and weight data using MinMax and Z-score normalization

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

```
np.random.seed(0)
heights = np.random.uniform(5.0, 6.5, 40)
weights = np.random.uniform(40000, 90000, 40)
data = pd.DataFrame({'Height': heights, 'Weight': weights})

scaler = MinMaxScaler(feature_range=(-10, 10))
data['Weight_MinMax'] = scaler.fit_transform(data[['Weight']])
```

```
z_scaler = StandardScaler()
data['Weight_Z'] = z_scaler.fit_transform(data[['Weight']])
data.head()
```

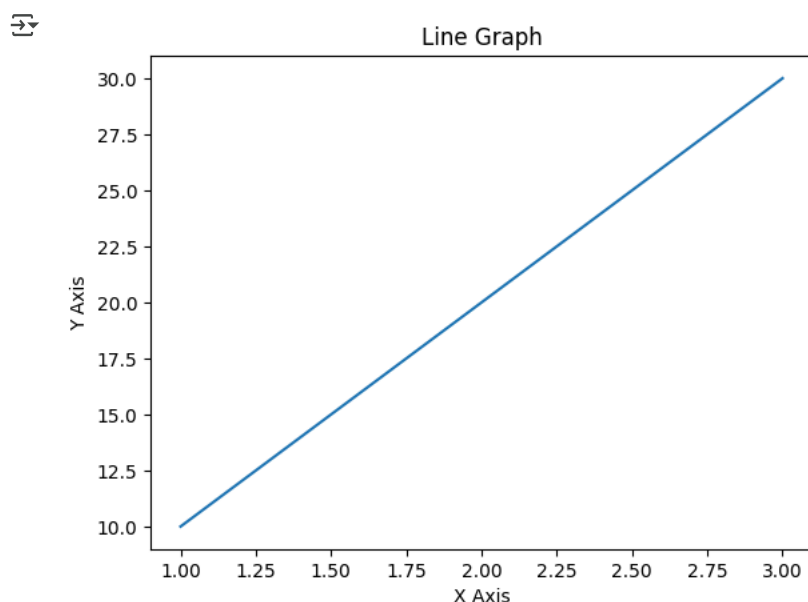
	Height	Weight	Weight_MinMax	Weight_Z
0	5.823220	57975.395029	-3.250636	-0.153235
1	6.072784	61851.597690	-1.617151	0.122899
2	5.904145	74881.559796	3.873853	1.051131
3	5.817325	43011.273581	-9.556722	-1.219253
4	5.635482	73338.335772	3.223518	0.941195

Next steps: [Generate code with data](#) [View recommended plots](#) [New interactive sheet](#)

Matplotlib Exercises

1. Write a Python program to draw a line with suitable label in the x axis, y axis and a title.

```
import matplotlib.pyplot as plt
plt.plot([1,2,3], [10,20,30])
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.title('Line Graph')
plt.show()
```



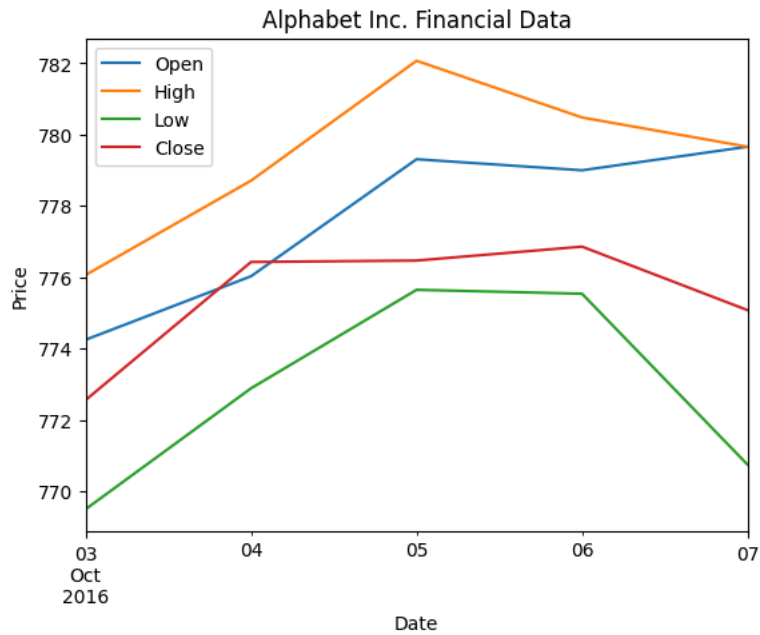
2. Write a Python program to draw line charts of the financial data of Alphabet Inc.

```
import pandas as pd
import matplotlib.pyplot as plt
from io import StringIO

csv_data = '''Date,Open,High,Low,Close
10-03-16,774.25,776.065002,769.5,772.559998
10-04-16,776.030029,778.710022,772.890015,776.429993
10-05-16,779.309998,782.070007,775.650024,776.469971
10-06-16,779.780.47998,775.539978,776.859985
10-07-16,779.659973,779.659973,770.75,775.080017'''

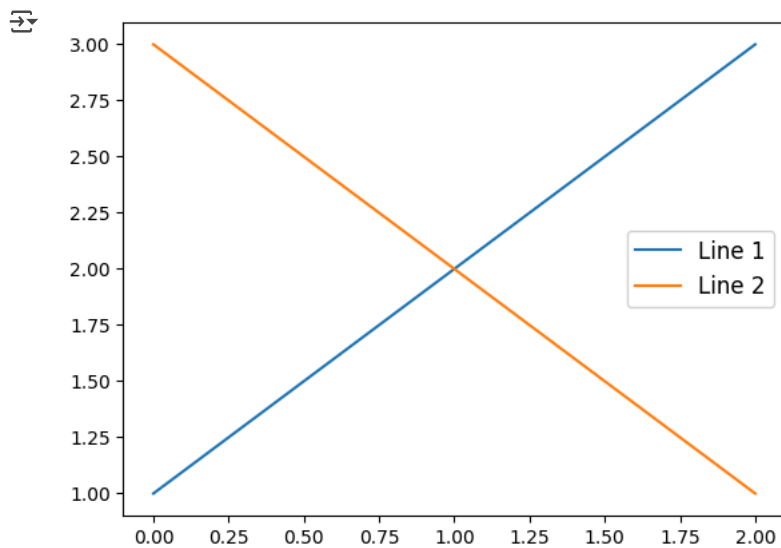
fdata = pd.read_csv(StringIO(csv_data), parse_dates=['Date'])
fdata.plot(x='Date', y=['Open', 'High', 'Low', 'Close'])
plt.title('Alphabet Inc. Financial Data')
plt.ylabel('Price')
plt.show()
```

 /tmp/ipython-input-25-3220148507.py:12: UserWarning: Could not infer format, so each element will be parsed individually
 fdata = pd.read_csv(StringIO(csv_data), parse_dates=['Date'])



✓ 3. Write a program to change the Change Legend Font Size in Matplotlib?

```
plt.plot([1, 2, 3], label='Line 1')
plt.plot([3, 2, 1], label='Line 2')
plt.legend(fontsize='large')
plt.show()
```



✓ 4. Write a program to change the legend position in Matplotlib

```
plt.plot([1, 2, 3], label='Line 1')
plt.plot([3, 2, 1], label='Line 2')
plt.legend(loc='upper left')
plt.show()
```

